

Language support for empty objects

ISO/IEC JTC1 SC22 WG21
P0840R0
Richard Smith
richard@metafoo.co.uk
2017-10-16

Motivation

Classes in generic code often wish to be handed a type from the user (such as an allocator) which will commonly be empty, and store an instance of it that will be used to customize the class's behavior. The generic code wishes to not pay to store the object in the case where it is empty. This is usually accomplished via the so-called "Empty Base Optimization" (EBO), whereby the class itself, or one of its data members, derives from the provided type, and the implementation reuses the "tail padding" of the empty type to store other portions of the class's data.

EBO introduces a number of problems, however:

- Limited applicability: EBO is not available for final classes, nor for classes with virtual bases that have non-public destructors.
- Name leakage: member names of base classes are visible to users of the derived class (unless shadowed), even if the base class is inaccessible. (Access checking is performed after the meaning of the name is identified.) In particular, unqualified lookups in code deriving from the class employing EBO is affected by names in the EBO base class.
- Implementation awkwardness: EBO requires state that would naturally be represented as a data member to be moved into a base class.

An approach without these problems would be preferred.

Proposal

We propose the addition of an attribute, `[[no_unique_address]]` to indicate that a unique address is not required for a non-static data member of a class. A non-static data member with this attribute may share its address with another object, if it could have zero size as a base class and the objects have distinct types.

Example:

```
template<typename Key, typename Value, typename Hash, typename Pred, typename Allocator>
class hash_map {
    [[no_unique_address]] Hash hasher;
    [[no_unique_address]] Pred pred;
    [[no_unique_address]] Allocator alloc;
    Bucket *buckets;
    // ...
public:
    // ...
};
```

Here, `hasher`, `pred`, and `alloc` could have the same address as `buckets` if their respective types are all empty.

Can we really use an attribute for this?

There is a lack of clarity concerning the problems for which an attribute is a reasonable solution. It is generally acknowledged that attributes cannot have arbitrary effects on the program semantics, but the precise reason why and its impact has less consensus.

I propose that the key constraint here is that of program portability: suppose a program uses a vendor-specific attribute, or a standard attribute from a later version of C++, or even a standard attribute that their implementation just doesn't implement yet. The result of compiling their program on that implementation should

still be a program that behaves correctly, according to the specification of the attribute. Therefore I propose the following criterion for determining whether an attribute is a valid approach to a problem:

compiling a valid program with all instances of a particular attribute ignored must result in a correct interpretation of the original program

This criterion is satisfied by the proposed `[[no_unique_address]]` attribute (and by all existing standard attributes).

It is worth noting that the above rule permits an attribute to affect the program's ABI, and we would expect the `[[no_unique_address]]` attribute to affect ABI, as it is intended to change struct layout.

Implementation concerns

In practice, implementations distinguish between objects with a known most-derived class type, and objects that might be base class subobjects in some cases. For an object that might be a base class subobject, implementations are careful not to touch the tail padding of the object, because the derived class might be reusing it for some other purpose. However, for an object with a known most-derived class, it is sometimes assumed that the tail padding "belongs" to the object and thus it is valid to widen a store into it.

Example:

```
struct HasTailPadding {
    HasTailPadding() : a(1), b(2) {}
    int a;
    char b;
};
```

When emitting a store to the `b` member of `HasTailPadding`, it may be profitable to use a 4-byte store instead of a 1-byte store (or even to use an 8-byte store to initialize the entire object). This is valid if the object is known to be of most-derived type `HasTailPadding`, but is otherwise not necessarily correct, as the store may overwrite some member of the derived class that was allocated into the tail padding.

This proposal would remove the ability for the compiler to perform such optimizations in the specific case where the class is empty. The author knows of no case where optimizers take advantage of this freedom for empty classes, however, so this is believed to be a purely theoretical problem.